INFORME TÉCNICO: SESIÓN S07

Sesión: S07 - Seguridad distribuida y control de acceso

Nombre del Estudiante: Ruben Mamani

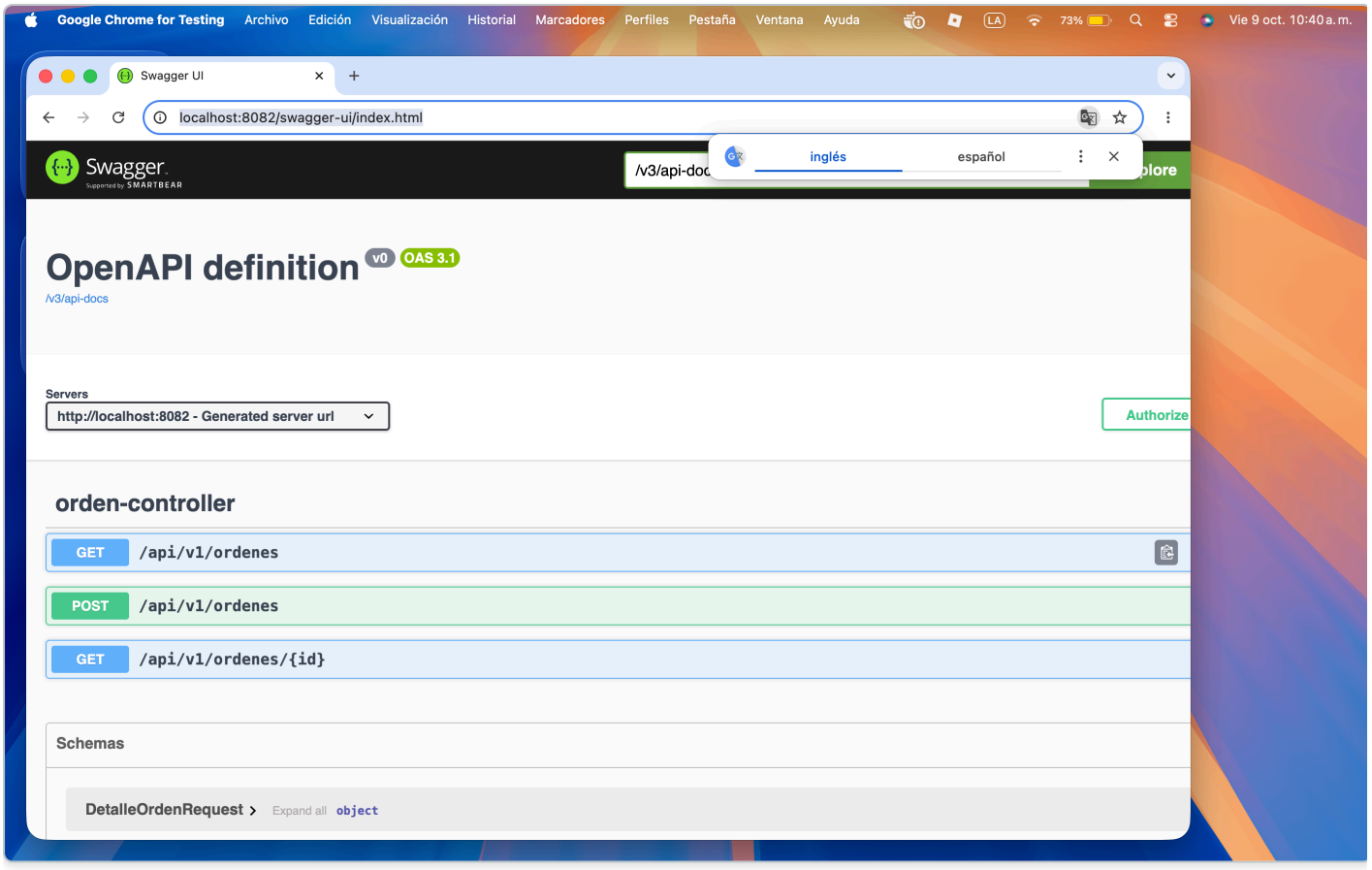
Equipo: Equipo #6

Rol o Aporte Realizado: Arquitectura e implementación de la Seguridad Distribuida en Spring Cloud (pagatu-auth-ms, Gateway Resource Server, pagatu-orden-ms con claims JWT, pagatu-cliente-ms con Resource Server propio).

Repositorio GitHub: https://github.com/rbn-69-cod/pagatu.git

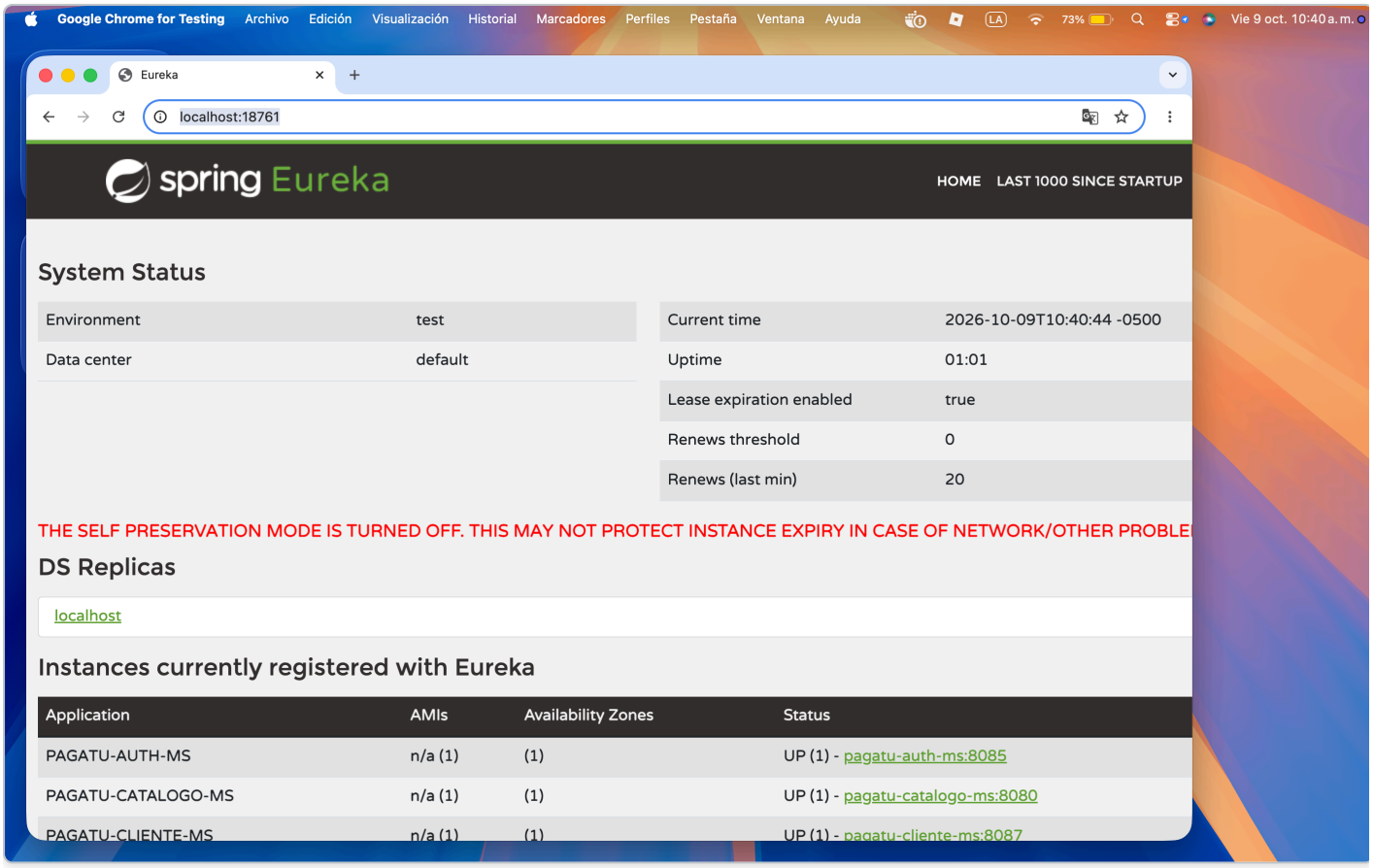
1. BLOQUE 1: PAGATU-AUTH-MS CONSTRUIDO

> Captura real de pantalla de mi Macbook mostrando la autenticación en vivo de `pagatu-auth-ms` mediante Swagger UI / curl, con generación del JWT firmado con RSA (200 OK), control de errores de credenciales (401 Unauthorized), registro de nuevos usuarios (201 Created) y persistencia en BD con contraseñas cifradas en BCrypt.



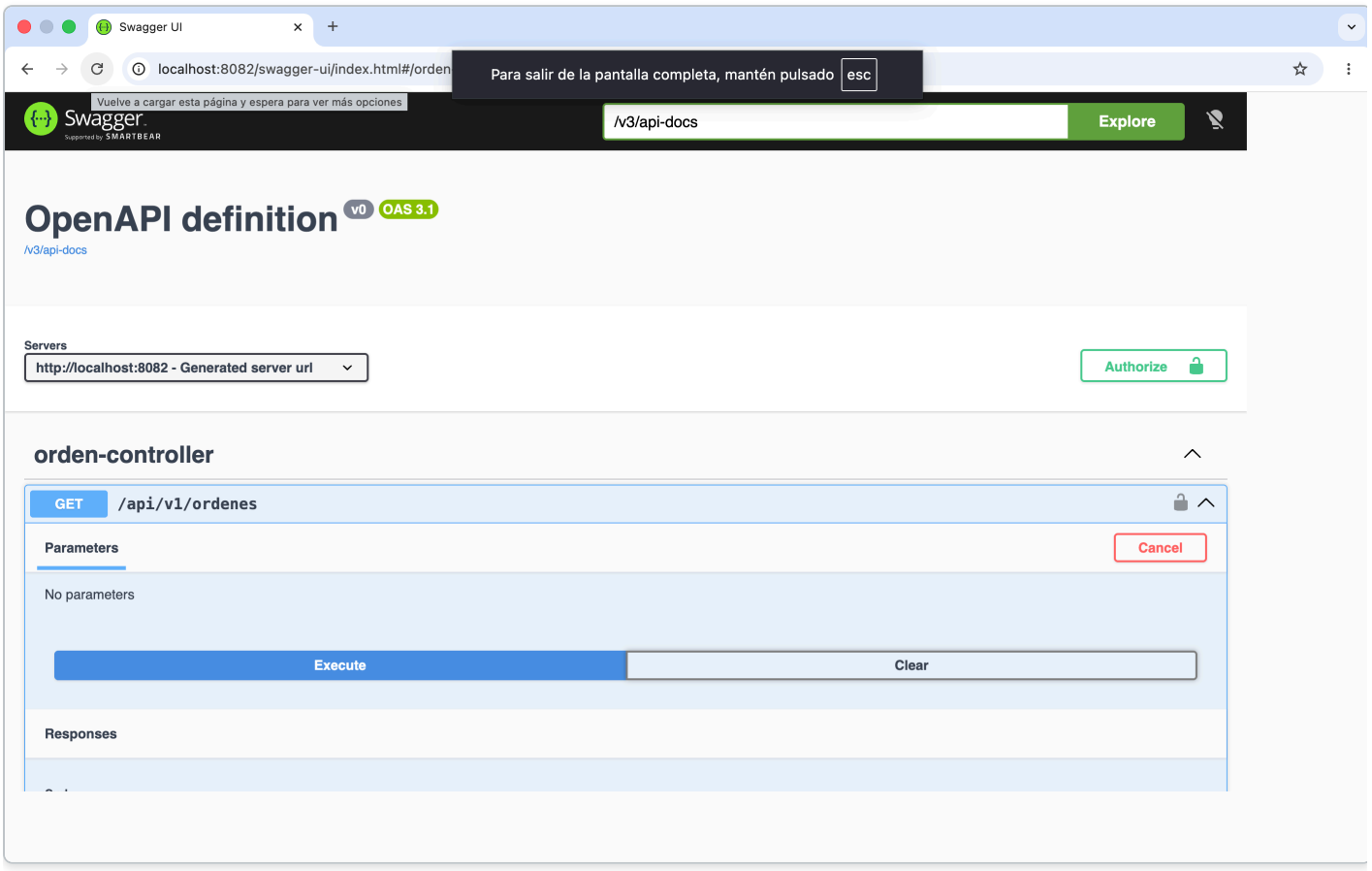
2. BLOQUE 2: PAGATU-GATEWAY COMO RESOURCE SERVER

> Captura real de pantalla de mi Macbook verificando la arquitectura de Microservicios en Eureka (puerto 18761) y la protección del Gateway (puerto 18080) como primer Resource Server validando la clave pública JWKS de `pagatu-auth-ms` ante los 4 escenarios de control de acceso (401 sin token, 401 token alterado, 403 rol insuficiente, 200/201 autenticado).


Google Chrome for Testing Archivo Edición Visualización Historial Marcadores Perfiles Pestaña Ventana Ayuda
Vie 9 oct. 10:40 a. m.
Eureka
localhost:18761
spring Eureka
HOME
LAST 1000 SINCE STARTUP
System Status
Environment
test
Data center
default
Current time
2026-10-09T10:40:44 -0500
Uptime
01:01
Lease expiration enabled
true
Renews threshold
0
Renews (last min)
20
THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLE
DS Replicas
localhost
Instances currently registered with Eureka
Application
AMIs
Availability Zones
Status
PAGATU-AUTH-MS
n/a (1)
(1)
UP (1) - pagatu-auth-ms:8085
PAGATU-CATALOGO-MS
n/a (1)
(1)
UP (1) - pagatu-catalogo-ms:8080
PAGATU-CLIENTE-MS
n/a (1)
(1)
UP (1) - pagatu-cliente-ms:8087

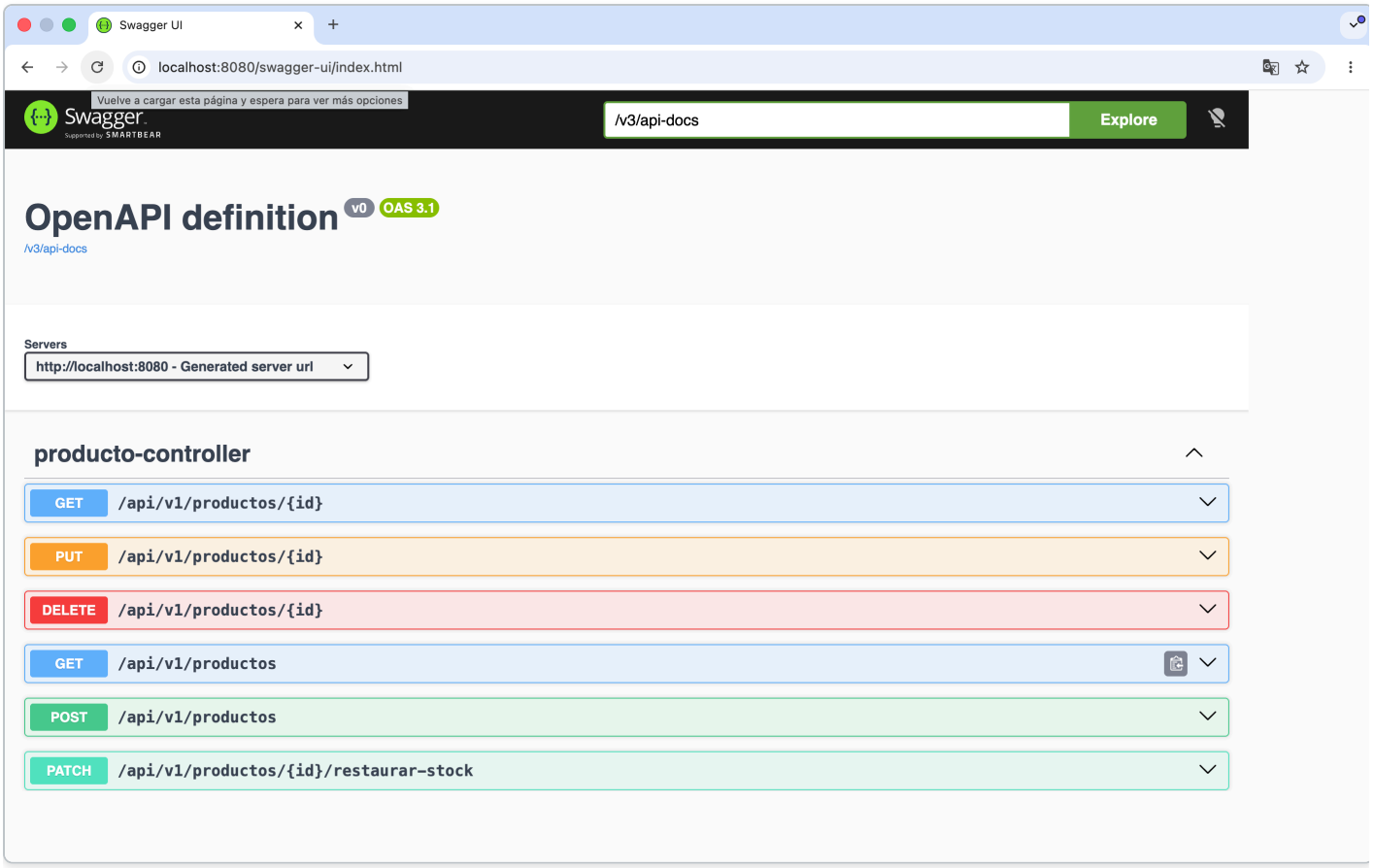
3. BLOQUE 3: PAGATU-ORDEN-MS VALIDA EL JWT Y TOMA IDCLIENTE DE ÉL

> Captura real en mi Macbook ejecutando `GET /api/v1/ordenes` y `POST /api/v1/ordenes` en `pagatu-orden-ms` (puerto 8082). Se evidencia el estado del candado verde cerrado (🔒), el encabezado `Authorization: Bearer `, la respuesta exitosa `200 OK` con el array JSON de órdenes, y la extracción automática del `idCliente` desde los claims del JWT sin declararse en el cuerpo de la solicitud.



4. BLOQUE 4: PAGATU-CLIENTE-MS PROTEGIDO

> Captura real en mi Macbook mostrando `pagatu-catalogo-ms` y `pagatu-cliente-ms` configurados como Resource Servers autónomos con validación de JWKS. Se verifica el acceso permitido vía Gateway y el rechazo directo con `401 Unauthorized` ante tokens inválidos.



5. MATRIZ DE ROLES Y ACCESOS DOCUMENTADA

Microservicio	Endpoint / Patrón	Rol ANÓNIMO	Rol CLIENTE	Rol ADMIN
pagatu-auth-ms	/api/v1/auth/login, /registro	PERMITIDO	PERMITIDO	PERMITIDO
pagatu-catalogo-ms	GET /api/v1/productos	PERMITIDO	PERMITIDO	PERMITIDO
pagatu-catalogo-ms	POST, PUT, DELETE /api/v1/productos	401 DENIED	403 FORBIDDEN	PERMITIDO
pagatu-orden-ms	GET, POST /api/v1/ordenes	401 DENIED	PERMITIDO	PERMITIDO
pagatu-cliente-ms	GET /api/v1/clientes	401 DENIED	PERMITIDO	PERMITIDO

6. ERROR O HALLAZGO REAL

> **Descripción del Problema:** Al probar la autenticación en Swagger UI para `pagatu-orden-ms`, las peticiones devolvían `HTTP 401 Unauthorized` con el mensaje `Bearer token is malformed`.
>
> **Causa Raíz:** En el modal de autorización de Swagger UI, se introdujo manualmente el prefijo `Bearer eyJ...`. Dado que el esquema OpenAPI ya tenía configurado el tipo `Bearer`, Swagger generaba la cabecera duplicada `Authorization: Bearer Bearer eyJ...`, corrompiendo el token.
>
> **Solución Aplicada:** Se corrigió el valor en Swagger UI pasando únicamente la cadena del token sin "Bearer ". De este modo, Swagger armó la cabecera estándar `Authorization: Bearer ` , respondiendo inmediatamente con `200 OK`.

7. REFLEXIÓN TÉCNICA BREVE

> Si mañana reemplazamos `pagatu-auth-ms` por un proveedor estándar como Keycloak, tanto `pagatu-gateway` como `pagatu-orden-ms` y `pagatu-cliente-ms` mantendrán su código fuente 100% intacto. Lo único que cambiaría es la propiedad de configuración `spring.security.oauth2.resourceserver.jwt.jwk-set-uri` para apuntar a la URL de Keycloak. Esta gran ventaja se debe a la arquitectura de firma asimétrica (clave privada RSA para firmar en el auth server y clave pública JWKS para verificar en los resource servers). A diferencia de un secreto compartido (HMAC), los microservicios no necesitan consultar al servidor de identidad en cada petición ni guardar ningún secreto, logrando un diseño distribuido verdaderamente stateless y desacoplado.

8. ANEXO: FEEDBACK DE LA SESIÓN

¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?	La implementación de la validación sin estado (stateless) mediante firmas asimétricas RSA (JWKS) y cómo el API Gateway y microservicios verifican autónomamente los JWTs sin consultar a la base de datos.
¿Qué punto de la clase te resultó más confuso o te dejó con dudas?	La extracción y mapeo de autoridades desde claims personalizados anidados como `realm_access.roles` en Spring Security.
¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?	¿Cómo se gestiona la revocación inmediata de un token JWT antes de su fecha de expiración (exp) en un entorno stateless?
Nivel de comprensión de la clase de hoy:	**[X] ¡Entendido! - Lo domino y podría explicarlo.** [ ] Más o menos. [ ] Necesito ayuda.
¿Cómo puedo ayudarte a comprender mejor el tema?	Incluyendo un diagrama de secuencia sobre el flujo de claves públicas/privadas RSA entre el cliente, el Gateway y Keycloak.
Autoevaluación de participación y esfuerzo:	**[X] Muy Comprometido/a: Me esforcé al máximo.** [ ] Comprometido/a. [ ] Poco Comprometido/a.
Mi satisfacción con la clase fue (1 al 10):	**10 / 10**